



VG2 Script Analysis: Comprehensive Findings

Summary

Reference document compiled after reading all five provided VG2 samples ([script_short.txt](#), [script_another.txt](#), [sql_script.txt](#), [script_from_vietnam.txt](#), [actual_script.txt](#)). Insights here will ground the architecture design later.



Section 1: The Real VG2 Format (revising assumptions from Phase 1)

1.1 Top-level structure — NOT XML

VG2 is a **flat, delimiter-separated text format**, not an XML tree as the documentation hinted. The actual on-disk structure is:

```
1 <OPTIONS>
2 /KEY1=value1
3 /KEY2=value2
4 /KEY3=value3
5 ...
6 </OPTIONS>
7 <body content – SQL or Python or utility directives>
8 <---- New Query ---->
9 <OPTIONS>
10 ...
11 </OPTIONS>
12 <body>
13 <---- New Query ---->
14 ...
```

Critical observations:

- Queries are separated by the literal string <---- New Query ---->
[\[script from vietnam | Txt\]](#)
- Options are slash-prefixed /KEY=value pairs inside an <OPTIONS> block
- **Format inconsistency alert:** [actual_script.txt](#) has the options inlined on the same logical "line" after <---- New Query ----> *without* explicit <OPTIONS>...</OPTIONS> markers — the parser must handle both layouts. [\[actual_script | Txt\]](#)
- Body content type is determined by which /KEY flags are present (see classification table below).

1.2 Query Block Classification (by /KEY signature)

Block Type	Signature Keys	Body Content	Sample
MARS Oracle read	/NODE=*.MARS, /OLEDB=SQLPlus, /ENGINE=VA, /RECORD=ViewName@version	SQL between /*BEGIN SQL*/...*/END SQL*/	[script another Txt] , [sql script Txt]
OASYS Oracle read	/NODE=*.OASYS (same engine flags)	SQL with @OASYSSCHEMA@ placeholder	[sql script Txt]
ARIES Oracle read	/NODE=<<>> (site macro) + /RECORD=AT_*@version	SQL with fully-qualified schema (e.g., ARIES_VIEWS.AV_dia_s ession)	[actual script Txt]
SQLite join/transf orm	/NODE=.\, /OLEDB=SQLite, /ENGINE=SQLite, /TABLE=file1,file2,..., /SQLITE_DT=...	Nested SELECT with /*L0*/.../*L1*/... markers	[sql script Txt] , [actual script Txt]
Write inline file	/WRITE-FILE=Y, /CSV=filename.{py,csv,bat,ht m}	Verbatim file content (Python, CSV, batch, HTML)	[script another Txt] , [script from viet nam Txt]
Run external utility	/UTILITIES=@EXEDIR@\name. {va,bat} args	empty (args are in the key)	[script another Txt]
Run inline utility macro	/UTILITIES={MACRO-NAME} "arg1" "arg2"	empty	[actual script Txt]
HTML report (3- stage)	/REPORT=HTML-RUN then HTML-LAYOUT then HTML- DELETE	Format/CSS spec; layout XML	[actual script Txt]
Cross- tab/pivot	/CTROW=..., /CTVALUE=..., /CTHEADER=..., /CTARRAY=alias,instance	SQL with CrossTab- >[[a0,instance;;Y]] marker	[actual script Txt]

1.3 Key OPTIONS attributes (the metadata vocabulary)

Key	Meaning	Translation note
/NODE	DB node — KM.[A15_PROD_21.].MARS, KM.OASYS, .\ (SQLite), <<>> (macro)	Drives reader selection
/UN=//, /PW=	Credentials (usually blank → integrated auth)	DataSyncX get_config_value(...)
/CSV	Output filename	Maps to writer destination

/TABLE	Comma-separated input file(s) for SQLite	SQLite TABLE registration
/HEADERS	Expected column names in output	Validate vs SQL aliases
/INSTANCE	SPF session ID — groups all blocks of one query design	Critical for understanding scope
/PROMPT-TEXT	Human-readable step description (e.g., "Step 1.1-a0. Fetching MARS Data")	Preserve as Python comments
/RECORD	LogicalViewName@version — the SPF logical view	Need view-expansion registry
/T=Yes/No	Terminal output flag	Mostly cosmetic
/TS=_YYYYMMDDHHMMSS	Timestamp suffix on output files	Preserve or deterministic-replace
/RESET=Y	Reset intermediate state	Cleanup hint
/DELETE=*Instance*	Cleanup pattern	Trigger unlink of *_{instance}.*
/SQLITE_DT	SQLite column types: (c)=char, (n)=number, (d)=date	Critical for pandas dtype mapping
/QUOTECSV=Y	Quote CSV values containing commas	csv.QUOTE_MINIMAL
/HEADERS_UNIQUE=Y	Dedupe duplicate column headers	DataFrame column dedup
/EMPTY_TO_NULL=Y	Empty strings → NULL	Map "" → pd.NA
/HADOOP_SERVER_DEFAULT, /APP_SERVER_DEFAULT	Compute backend defaults	Inform DataSyncX reader selection
/WORKDIR=.\	Working directory	Set os.chdir() or path prefix
/OLEDB, /ENGINE	Driver + engine pair (SQLPlus/VA for Oracle, SQLite/SQLite for local)	Reader dispatch
/UTILITIES	Inline utility command	Map to Python equivalent
/REPORT	HTML-report sub-stage	3-stage report builder
/CTROW, /CTVALUE, /CTHEADER, /CTARRAY	Cross-tab/pivot spec	pd.pivot_table()
/OUTLOOK=N	Don't open Outlook on completion	Cosmetic
/JSON-ONLY=N	HTML vs JSON output	Report config
/CHART-INSTANCE	Chart sub-instance ID	Internal tracking

Section 2: SQL & Body-Content Patterns

2.1 The "@[]@" placeholder confirmation

Every MARS read uses the pattern:

```

1 FROM @[]@F_LotHist f0
2 LEFT JOIN @[]@F_Product p ON ...

```

This **confirms** the verified DataSyncX behavior from [datasyncx_reader_fact.txt](#): DataSyncX's query_mao() substitutes @[]@ with the hardcoded schema based on the site (e.g., KM → A15_PROD_21). **Translator must NOT pre-substitute — pass through as-is to MarsReader.** [\[datasyncx ...eader fact | Txt\]](#)

2.2 OASYS schema placeholder

OASYS uses a different placeholder: @OASYSSCHEMA@ (e.g., @OASYSSCHEMA@P_SPC_Batch_Lot v1). Unlike MARS, DataSyncX's OracleReader does **not** auto-substitute. Translator strategies: [\[sql_script | Txt\]](#)

- Option A: Pre-substitute using a hand-maintained {site → oasys_schema} map
- Option B: Use a custom SQL preprocessor in a Processor step
- Option C: Document as a deployment-time config

2.3 ARIES uses fully-qualified schemas

ARIES SQL uses explicit prefixes:

```

1 FROM ARIES_VIEWS.AV_dia_session z0
2 LEFT JOIN ARIES_VIEWS.AV_dia_media_testing z2 ON ...

```

And also bare-table names (e.g., A_Testing_Session ats, A_MARS_Lot ml) inside the same query. No substitution needed — pass through to AriesReader / XeusReader. [\[actual_script | Txt\]](#)

2.4 The SQL_Get_CSV_List macro — a major translation challenge

Repeatedly seen pattern:

```

1 WHERE (f0.lot In SQL_Get_CSV_List(".\yeuchuan_a0_29397.tab",
lot, "f0.lot In"))

```

This SPF-specific macro reads a column from a CSV at SQL-build time and emits an IN clause. Appears in nearly every multi-step script. [\[sql_script | Txt\]](#), [\[actual_script | Txt\]](#)

Translation approach:

1. Detect the macro pattern (regex on SQL_Get_CSV_List(...\))
2. At runtime, read the referenced CSV with pandas
3. Build the IN ('val1', 'val2', ...) string
4. **Chunk into 1000-element batches** (Oracle's hard limit) — SPF does this automatically; we must replicate it
5. Inject before passing to the reader

2.5 Multi-level computed columns (/L0*/.../L1*/.../Ln*/)

The most complex SQL pattern, seen in [actual_script.txt](#):

```

1  SELECT /*L3*/ DISTINCT ..., DENSE_RANK() OVER (...) FROM (
2    SELECT /*L2*/ ..., CASE WHEN ... AS Flag FROM (
3      SELECT /*L1*/ ..., CASE WHEN sub_plane_x NOT BETWEEN ... AS
Set_Limit_X FROM (
4        SELECT /*L0*/ a1.col, a0.col, CASE WHEN '' THEN NULL ELSE
CAST(...) END AS sub_plane_x
5        FROM [tableA] a1 LEFT JOIN [tableB] a0 ON ...
6      ) t /*L0*/
7    ) t /*L1*/
8  ) t /*L2*/
9  WHERE [Flag] = 'flag'

```

This directly mirrors what

[Introduction_to_SQLPathFinder_SelfPaced_Computed_Columns_2.ppsm](https://intel-my.sharepoint.com/personal/yeu_chuan_lim_intel_com/_layouts/15/Doc.aspx?sourcedoc=%7B42FA75A1-BC51-4DEA-9F74-

6A7CFC17B8B2%7D&file=Introduction_to_SQLPathFinder_SelfPaced_Computed_Columns_2.ppsm&action=edit&mobileredirect=true&EntityRepresentationId=4eeff3b5-20f2-4e0d-a19d-a6269d8e0067) described as *****"Level" in the Columns Grid****. **【6-afa4b6】** Each L_n is a successive computed-column layer.

Translation strategies:

Strategy	Pros	Cons
A. Pass SQL to embedded SQLite	Faithful, no logic rewrite	Hard to debug, opaque to user
B. Decompose into chained Processors	Each level = one Processor, transparent	More complex codegen; need expression parser
C. Hybrid: bottom levels in SQLite, top in pandas	Balances faithfulness + flexibility	Two engines to maintain

→ Recommendation: **Strategy B** with a fallback to A for unsupported expressions.

2.6 Cross-Tab / Pivot pattern

In [actual_script.txt](#), a cross-tab block:

```

1  /CTROW=facility,lot,operation,test_end_date,...
2  /CTVALUE=Sub_plane
3  /CTHEADER=test_name
4  /CTARRAY=a0,15507

```

And in the SELECT body: CrossTab->[[a0,15507;;N]] (or :Y) as a placeholder showing where the pivot expansion goes. [\[actual_script | Txt\]](#)

Translation: `pd.pivot_table(index=CTROW.split(','), columns=CTHEADER, values=CTVALUE, aggfunc='max')` — the `Max(Sub_plane)` aggregate in the surrounding `SELECT` confirms `aggfunc`.

2.7 SPF macro variable references

Three syntactic forms encountered:

Syntax	Meaning	Example
<code><<<varname>>></code>	User-defined or system global	<code><<<SFOLDER>>></code> , <code><<<spfinstance>>></code>
<code><<>></code>	Positional macro placeholder (inside {START-MACRO})	<code>/NODE=<<>></code> — site comes from macro CSV
<code>VAR(<<<var>>>)</code>	Explicit reference in IF-THEN tests	<code>VAR(<<>>) EQS FAIL</code>

Critical: `<<>>` is **order-dependent** — it refers to the *next* column in the macro CSV's current row. Translator must track macro context AND column position. This is a major source of correctness bugs.

2.8 Oracle-specific syntax to preserve

Pattern	Notes
<code>/*+ index(c0) */</code>	Oracle optimizer hint — keep as-is, harmless to DataSyncX
<code>To_Char(date, 'yyyy-mm-dd hh24:mi:ss')</code>	Standard SPF date formatting
<code>TRUNC(SYSDATE) - 365</code>	Date arithmetic — keep as Oracle SQL
<code>NVL(col, default)</code>	Oracle NULL coalesce
<code>decode(col, val1, ret1, default)</code>	Oracle conditional
<code> </code> for string concat	Standard Oracle/SQLite syntax
<code>-- Tail A (and similar)</code>	Leftover SPF design markers — preserve as comments for traceability

Section 3: Utility & Macro Patterns

3.1 Inline utility macros (curly-brace form)

Found in [actual_script.txt](#): [\[actual_script | Txt\]](#)

Macro	Args	Translation
<code>{START-MACRO} "file.csv" "Y/N"</code>	CSV file + skip-header flag	for row in <code>pd.read_csv(file).itertuples():</code>
<code>{END-MACRO}</code>	—	Close the for loop
<code>{IF-THEN} "var" "OP" "val" "AND/OR" "var2" "OP2" "val2"</code>	up to 2 conditions	if {expr}:
<code>{ELSE}</code>	—	else:
<code>{END-IF}</code>	—	Close the if

{ROWS-IN-FILE} "file" "VARIABLE" "extra"	File + env-var to set	count = sum(1 for _ in open(file)) - 1
---	--------------------------	---

3.2 External tools invoked via /UTILITIES=@EXEDIR@\...

Tool	Real behavior	Python equivalent
SPFCopy.bat "src" "dst" "N"	File copy with optional overwrite check	shutil.copy2()
SPFDelete.bat "f1,f2,f3" "N"	Multi-file delete	pathlib.Path(f).unlink() per file
SPFRename.va "old" "new"	File rename	os.rename()
RoboCopy.va "file" "dst" "src" "retries" "wait" ...	Robust network copy	Custom retry loop with shutil
SQLPathFinder_Email.va "attachments" "to" "subject" "body" "cc" ... "N" "N"	SMTP email	smtplib or DataSyncX notification
Run_Python_Script.va "script.py" "" "N" "atd_atm.hadoop" "Python-v3"	Execute Python on remote/local	Direct exec or subprocess.run()

3.3 The "embedded Python" pattern (the easy win)

[script_another.txt](#) and [script_from_vietnam.txt](#) both follow this idiom:

1. Run an SPF query → produces a CSV [\[script_another | Txt\]](#)
2. /WRITE-FILE=Y /CSV=lich.py → emit a Python file inline [\[script_another | Txt\]](#)
3. /UTILITIES=@EXEDIR@\Run_Python_Script.va "lich.py" ... → execute it [\[script_another | Txt\]](#)

This is the easiest case to translate — the Python is already written; just preserve it and run it directly as part of the generated pipeline (or merge it inline with the DataSyncX flow).

3.4 The HTML report 3-stage pattern (complex, low-priority)

[actual_script.txt](#) shows HTML-RUN → HTML-LAYOUT → HTML-DELETE sequence with CSS, format specs, and layout tables. Likely **out of scope for v1** — flag as # TODO: HTML report — manual port required. [\[actual_script | Txt\]](#)

Section 4: Cross-Sample Patterns (the recurring "shape" of a VG2 script)

Looking across all five samples, every non-trivial script follows roughly this shape:

1
7

```

2 | Phase 1: SETUP
  |
3 |   • Write config files inline (/WRITE-FILE for path.csv etc.)
  |
4 |   • Copy reference files from network shares
  |
5 |
  |-----
  |
6 | Phase 2: PRIMARY EXTRACTS (1..N parallel sources)
  |
7 |   • MARS read → a0.tab
  |
8 |   • OASYS read → a1.tab (often filters using a0 via
SQL_Get...) |
9 |   • ARIES read → a2.tab
  |
10 |
  |-----
  |
11 | Phase 3: JOIN + COMPUTED COLUMNS (in SQLite)
  |
12 |   • SELECT /*L0*/ JOIN of a0/a1/a2 with CASE/CAST
  |
13 |   • SELECT /*L1*/ adds Set_Limit_* flags
  |
14 |   • SELECT /*L2*/ adds Flag
  |
15 |   • SELECT /*L3*/ adds DENSE_RANK
  |
16 |   • WHERE [Flag] = 'flag'
  |
17 |
  |-----
  |
18 | Phase 4: TRANSFORM + ANALYZE (inline Python via
Run_Python...) |
19 |   • pandas merge / pivot_table / np.where for color flags
  |
20 |   • Group-by aggregations (volume sum, yield calc)
  |

```

```

21 | • Write final CSVs to dashboard folder
    |
22 |
    |_____
23 | Phase 5: DISTRIBUTE + ALERT
    |
24 | • SPFCopy to network share
    |
25 | • Email on conditional triggers
    |
26 |
    |_____
    |

```

This pattern is **stable** across [sql_script.txt](#), [script_from_vietnam.txt](#), and [actual_script.txt](#) — and the design should optimize for it.

4.1 The "instance" grouping

All blocks within a logical query design share the same /INSTANCE=<number>. Examples:

- script_short.txt → instance 8486 (1 block) [\[script_short | Txt\]](#)
- script_another.txt → instance 8486 (3 blocks) [\[script_another | Txt\]](#)
- sql_script.txt → instance 29397 (3 blocks) [\[sql_script | Txt\]](#)
- script_from_vietnam.txt → instance 29397 (14+ blocks) [\[script_from_vietnam | Txt\]](#)
- actual_script.txt → instance 15507 (40+ blocks) [\[actual_script | Txt\]](#)

Hence /INSTANCE is essentially the **module identifier** — translator should use it for output script naming and file-prefix grouping (e.g., yeuchuan_a0_29397.tab = MARS alias a0's output for instance 29397).

4.2 Filename conventions

Pattern: <user>_<alias>_<instance>.<ext>

- yeuchuan_a0_29397.tab — user's MARS alias a0 output [\[sql_script | Txt\]](#)
- yeuchuan_a1_29397.tab — user's OASYS alias a1 output
- yeuchuan_SQL_15507.tab — SQLite intermediate output [\[actual_script | Txt\]](#)

Preserve in translation for traceability OR replace with semantic names from /PROMPT-TEXT.

⚠ Section 5: Failure Points & Risk Inventory

Cross-referenced against Phase 1's docs/guides findings.

#	Risk	Severity	Source observation	Mitigation strategy
1	VG2 format inconsistency — some scripts use <OPTIONS>...</OPTIONS>, others bare key lines	High	Comparing script_short.txt vs actual_script.txt [script_short Txt] , [actual_script Txt]	Parser must handle both with a normalization layer
2	SPF logical view expansion gap — VG2 only carries physical tables + selected columns, NOT intra-view JOINS/implicit WHEREs/bridge sub-aliases	High	Confirmed in datasyncx_reader_fact.txt [datasyncx ...reader fact Txt] ; SPF SQL we see already includes these JOINS (e.g., the LEFT JOIN @[]@F_Product after @[]@F_LotHist)	Hand-maintained view-expansion YAML registry
3	SQL_Get_CSV_List macro — no DataSyncX equivalent	High	Seen in every multi-step query [sql_script Txt] , [actual_script Txt]	Build Python preprocessor that reads CSV → chunked IN clause, injects before reader call. Watch Oracle 1000-element limit
4	@OASYSSCHEMA@ substitution — OracleReader doesn't auto-substitute	Medium	Seen in sql_script.txt [sql_script Txt]	Pre-substitute via site→schema map OR custom processor
5	<<>> positional macro inside {START-MACRO} is column-order-dependent	High	Throughout actual_script.txt [actual_script Txt]	Track macro context as a stack; resolve <<>> references relative to current macro CSV's columns. Easy to get wrong silently
6	Multi-level computed columns (/L0*/.../Ln*/) need decomposition	Medium	actual_script.txt Step 9 has 4 levels [actual_script Txt]	Mini SQL expression parser per level OR pass-through to embedded SQLite/DuckDB
7	SQLite integer division gotcha — 5/2 = 2 not 2.5	Low	Flagged in computed-columns doc [Introducti... Columns_2 PowerPoint] ;	Mostly handled by users; verify in CAST-emitter

			samples already use CAST(... AS REAL) defensively [actual_script Txt]	
8	HTML report blocks are complex and SPF-specific	● Low	actual_script.txt Step 1-1 [actual_script Txt]	Out of scope for v1 — emit # TODO marker
9	External .va/.bat utilities are a long tail	● Medium	Multiple in actual_script.txt [actual_script Txt]	Build adapter library for known tools; fail loudly on unknown
10	/INSTANCE and timestamp suffixes affect file naming/uniqueness	● Low	Throughout samples	Decide policy: preserve original OR semantic-rename
11	MARS vs OASYS detection from VG2 alone — both are Oracle	● Medium	Need heuristic from datasyncx_reader_fact.txt [datasyncx ...eader fact Txt]	Look at /NODE=*.MARS vs /NODE=*.OASYS first; fall back to view-prefix heuristic (F_* vs P_*)
12	ARIES queries use /NODE=<<>> (site comes from enclosing macro)	● Medium	actual_script.txt Step 5-1.1-a0 [actual_script Txt]	Translator needs macro-context-aware reader instantiation
13	Inlined Python may have its own bugs — translator must not "fix" it	● Low	script from vietnam.txt has 100+ lines of inline pandas [script from vietnam Txt]	Preserve verbatim; offer separate refactoring tool later
14	Dataflow ordering across blocks — block N often reads CSV written by block N-1	● High	Every multi-block sample	Emit blocks sequentially OR build a DAG with CSV dependencies as edges
15	HEADERS/HEADERS_UNIQUE mismatches with SQL AS aliases	● Medium	actual_script.txt has /HEADERS=... listing columns that don't all appear in SELECT (e.g., Set_Limit_plane_X appears at L2 but is in HEADERS) [actual_script Txt]	Validate but tolerate; emit warnings
16	/RESET=Y and /DELETE=*Instance* are cleanup hints with ordering	● Low	sql_script.txt [sql_script Txt]	Map to explicit pathlib.Path.glob().unlink()
17	Cross-tab placeholder CrossTab->[[a0,15507;;Y]] must be expanded inline in SQL	● Medium	actual_script.txt Step 8 [actual_script Txt]	Special expander: replace marker with the pivoted column expressions OR drop into a pandas

				pivot_table processor stage
18	/CTROW lists may include columns that need to first be selected — dependency from CTROW columns to underlying SELECT	● Medium	Same as above	Build column-resolution pass before SQL emission
19	Date format string mismatch between Oracle and pandas	● Low	'yyyy-mm-dd hh24:mi:ss' vs '%Y-%m-%d %H:%M:%S'	Translation map for the few common formats
20	The path.csv indirection pattern — config file specifies network paths	● Medium	script from vietnam.txt writes path.csv then reads paths from it in subsequent Python [script from vietnam Txt]	Honor the pattern OR lift paths to a DataSyncX .env/config



Section 6: Sample-by-Sample Quick Reference

6.1 [script_short.txt](#) — minimal SQLite query

- **1 block**, instance 8486
- SQLite SELECT DISTINCT of one column from a pre-existing CSV (ww_yield.csv) [\[script_short | Txt\]](#)
- **Translation target:**
pd.read_csv('ww_yield.csv')['owner'].drop_duplicates().to_csv('owner.csv', index=False)
- **Value as test case:** smallest possible — good for parser smoke test

6.2 [script_another.txt](#) — MARS → Python → run pattern

- **3 blocks**, instance 8486
- Block 1: MARS calendar query writes calendar_ref.csv [\[script_another | Txt\]](#)
- Block 2: /WRITE-FILE=Y emits lich.py (pandas date manipulation)
- Block 3: Run_Python_Script.va lich.py executes it
- **Value as test case:** demonstrates the canonical MARS+Python idiom

6.3 [sql_script.txt](#) — MARS + OASYS + SQLite join

- **3 blocks**, instance 29397
- Block 1: MARS F_LotHist + F_Product JOIN [\[sql_script | Txt\]](#)
- Block 2: OASYS multi-table JOIN (P_SPC_Batch_Lot v1, P_SPC_Batch v2, P_SPC_Session v3) filtered by Block 1's lots via SQL_Get_CSV_List

- Block 3: SQLite LEFT OUTER JOIN of the two outputs into final
- **Value as test case:** demonstrates MARS+OASYS cross-source pattern with SQL_Get_CSV_List and the **OASYS multi-alias bridge problem** flagged in [datasyncx_reader_fact.txt](#) (v2 is a bridge between v1 and v3 but appears in neither HEADERS nor selected columns) [\[datasyncx ...eader_fact | Txt\]](#)

6.4 [script_from_vietnam.txt](#) — full production dashboard pipeline

- ~14 blocks, instance 29397
- Phases: write config → MARS extract → inline Python (date prep) → run Python → embedded Python (200+ lines of yield analysis) → run → multiple SQLite distinct-extractions for filter dropdowns → final Excel composition [\[script_from_vietnam | Txt\]](#)
- Uses path.csv indirection pattern
- Has nested pandas logic with np.where, pivot_table, complex multi-column merges
- **Value as test case:** real-world production complexity; pandas-heavy

6.5 [actual_script.txt](#) — the most complex, with macros + 4-level computed columns + cross-tab

- ~40+ blocks, instance 15507
- Contains: HTML report 3-stage, START-MACRO/END-MACRO blocks, IF-THEN/ELSE chains, ROWS-IN-FILE, dummy file creation, RoboCopy, multiple emails on alert conditions, MARS + ARIES reads, cross-tab pivot, 4-level computed column nesting (L0→L1→L2→L3 with DENSE_RANK), final SQLite filter [\[actual_script | Txt\]](#)
- Uses <<<SFOLDER>>>, <<>> positional macros, and VAR(<<<...>>>) references
- **Value as test case:** this is the **stress test** — if the translator handles this, it handles anything

Section 7: Refined Architecture Implications (preview for next planning step)

Based on these samples, the translator needs at minimum:

1. **A two-phase parser:**
 - Phase A: Tokenize on <---- New Query ----> and <OPTIONS>...</OPTIONS> boundaries (with fallback for the no-marker format in actual_script.txt)
 - Phase B: For each block, dispatch on the /KEY signature to the right block-type AST node
2. **A block-type registry (Strategy pattern)** with at least these emitters:
 - MarsReadEmitter
 - OasysReadEmitter

- AriesReadEmitter
 - SqliteJoinEmitter (handles L0..Ln decomposition)
 - WriteFileEmitter (verbatim passthrough, with macro-var resolution for <<<...>>>)
 - UtilityEmitter (dispatches on inline-macro name OR external-tool path)
 - CrossTabEmitter
 - HtmlReportEmitter (stub for v1)
3. **A macro-context manager** for tracking nested {START-MACRO} scopes and resolving <<>> positional references
 4. **A view-expansion registry** (YAML) that the MARS/OASYS/ARIES emitters consult before emitting reader calls (per the [datasyncx_reader_fact.txt](#) requirement)
[\[datasyncx ...eader fact | Txt\]](#)
 5. **A SQL preprocessor** that handles SQL_Get_CSV_List expansion with chunking
 6. **A dataflow graph builder** that orders blocks by CSV produces/consumes edges
 7. **An adapter library** for external .va/.bat utilities
-

☑ Confidence & Open Items

Area	Confidence	Next step
VG2 syntactic structure	High	Lock in parser strategy
Block-type taxonomy	High	Build the dispatch table
MARS/OASYS/ARIES → reader mapping	High	Reference datasyncx_reader_fact.txt [datasyncx ...eader fact Txt]
SQL_Get_CSV_List semantics	Medium-High	Need to verify chunking behavior with one real run
Multi-level (L0..Ln) decomposition rules	Medium	Decide Strategy A vs B vs C (Section 2.5)
Cross-tab CrossTab->[[...]] placeholder expansion mechanics	Medium	Need 1-2 more pivot examples to lock pattern
HTML report pipeline	Low (intentional)	Defer to v2
Macro <<>> positional resolution edge cases (nested macros)	Medium	Need at least one nested-macro sample

I'm ready to use these findings to lay out the **system architecture** (module structure, class diagram, design-pattern choices) when you give the go-ahead. Or, if useful, I could also record this analysis to memory so the context persists across future planning sessions — just let me know.